



Load Data into a DynamoDB Table



Table: ContentCatalog - Items returned (6)

Scan started on December 11, 2025, 12:00:39



Actions

Create item

< 1 > ⚙

☐	<i>Id (Number)</i>	<i>Authors</i>	<i>ContentType</i>	<i>Difficulty</i>	<i>Price</i>	<i>ProjectCategory</i>
☐	3	[{"S": "Ne...	Project	Easy peasy	0	AI/ML
☐	2	[{"S": "Ne...	Project	Easy peasy	0	Analytics
☐	203		Video		0	
☐	202		Video		0	
☐	201		Video		0	
☐	1	[{"S": "Nat...	Project	Easy peasy	0	Storage



Introducing Today's Project!

What is Amazon DynamoDB?

Amazon DynamoDB is a fully managed NoSQL database service that provides fast, predictable performance and automatic scalability. It stores data in key value and document formats and removes the need to manage servers, replication, backups, or hardware provisioning.

How I used Amazon DynamoDB in this project

In today's project, I used Amazon DynamoDB to create a table with Amazon DynamoDB, upload data into DynamoDB using AWS CloudShell and edit data in DynamoDB tables.

One thing I didn't expect in this project was...

One thing I didn't expect in this project is encounter error while trying to view data in the table.

This project took me...

This project took me 2 hours.



Create a DynamoDB table

DynamoDB tables organize data using primary keys for item uniqueness, partitions for physical storage distribution, and indexes (LSIs and GSIs) for alternative query patterns.

An attribute is a single piece of data that describes an item.

The screenshot shows the AWS DynamoDB console interface. At the top, it says 'Table: NextWorkStudents - Items returned (1)'. Below this, it indicates 'Scan started on December 10, 2025, 22:00:14'. There are buttons for 'Actions' and 'Create item'. A table below shows the data returned:

☐	StudentName (String)	ProjectsComplete
☐	Nikko	4



Read and Write Capacity

Read capacity units (RCUs) and write capacity units (WCUs) are the units that determine how much read and write capacity a DynamoDB table can handle, based on item size and consistency requirements.

Amazon DynamoDB's Free Tier covers 25 GB of storage, plus up to 25 RCUs and 25 WCUs each month at no cost. I turned off auto scaling because I wanted to control capacity manually and avoid unexpected throughput increases that could raise costs.

The screenshot shows the 'Read/write capacity settings' page in the AWS Management Console. It is divided into two main sections: 'Read capacity' and 'Write capacity'. Each section has an 'Auto scaling' toggle and a 'Provisioned capacity units' input field. In the 'Read capacity' section, 'Auto scaling' is set to 'Off' and 'Provisioned capacity units' is set to '1'. In the 'Write capacity' section, 'Auto scaling' is also set to 'Off' and 'Provisioned capacity units' is set to '1'. The 'Capacity mode' at the top is set to 'Provisioned'.

Read/write capacity settings [info](#)

Capacity mode

☐ On-demand
Simplify billing by paying for the actual reads and writes your application performs.

☒ **Provisioned**
Manage and optimize your costs by allocating read/write capacity in advance.

Read capacity

Auto scaling [info](#)
Dynamically adjusts provisioned throughput capacity on your behalf in response to actual traffic patterns.

☐ On

☒ **Off**

Provisioned capacity units

1

Write capacity

Auto scaling [info](#)
Dynamically adjusts provisioned throughput capacity on your behalf in response to actual traffic patterns.

☐ On

☒ **Off**

Provisioned capacity units

1



Using CLI and CloudShell

AWS CloudShell is a browser-based shell that provides a pre-configured Linux environment with the AWS CLI and tools already installed, allowing you to run commands against your AWS resources without local setup.

AWS CLI is a tool that provides programmatic access to AWS APIs through terminal commands, supporting automation, scripting, and resource management.

I ran a CLI command in AWS CloudShell that created tables.

```
CloudShell
100-42081-1
>
> --provisioned-throughput \
>   ReadCapacityUnits=1,WriteCapacityUnits=1 \
>   --query "TableDescription.TableStatus"
"CREATING"
$ aws dynamodb create-table \
  --table-name Post \
  --attribute-definitions \
  --attribute-name-forumname AttributeTypes=5 \
  --attribute-name-subject AttributeTypes=5 \
  --key-schema \
  --attribute-name-forumname KeyType=HASH \
  --attribute-name-subject KeyType=RANGE \
  --provisioned-throughput \
  ReadCapacityUnits=1,WriteCapacityUnits=1 \
  --query "TableDescription.TableStatus"
"CREATING"
$ aws dynamodb create-table \
  --table-name Comment \
  --attribute-definitions \
  --attribute-name-id AttributeTypes=5 \
  --attribute-name-commentdatetime AttributeTypes=5 \
  --key-schema \
  --attribute-name-id KeyType=HASH \
  --attribute-name-commentdatetime KeyType=RANGE \
  --provisioned-throughput \
  ReadCapacityUnits=1,WriteCapacityUnits=1 \
  --query "TableDescription.TableStatus"
"CREATING"
$
$
$
```



Loading Data with CLI

I ran a CLI command in AWS CloudShell that retrieved a ZIP file of sample data, extracted it, and then populated my DynamoDB tables by running batch-write-item for each JSON dataset.

```
~ $ aws dynamodb batch-write-item --request-items file://ContentCatalog.json
{
  "UnprocessedItems": {}
}
~ $ aws dynamodb batch-write-item --request-items file://Forum.json
{
  "UnprocessedItems": {}
}
~ $ aws dynamodb batch-write-item --request-items file://Post.json
{
  "UnprocessedItems": {}
}
~ $ aws dynamodb batch-write-item --request-items file://Comment.json
{
  "UnprocessedItems": {}
}
~ $
```



Observing Item Attributes

The screenshot shows a web interface titled "Attributes" with a table of attributes. The table has columns for "Attribute name", "Value", "Type", and "Remove". The attributes listed are:

Attribute name	Value	Type	Remove
☒ Id - Partition key	1	Number	
☒ Authors	Insert a field	List	Remove
☐ ContentType	Project	String	Remove
☐ Difficulty	Easy peasy	String	Remove
☐ Price	0	Number	Remove
☐ ProjectCategory	Storage	String	Remove
☐ Published	☒ True ☐ False	Boolean	Remove
☐ Title	Host a Website on Amazon S3	String	Remove
☐ URL	aws-host-a-website-on-s3	String	Remove

At the bottom right of the form are three buttons: "Cancel", "Save", and "Save and close".

I checked a ContentCatalog item, which had the following attributes: Id, Author, ContentType, Difficulty, Price, ProjectCategory, Published, Services, Title and URL.

I checked another ContentCatalog item, which had a different set of attributes ContentType, price, Services, Title, URI and VideoType.



Benefits of DynamoDB

A benefit of DynamoDB over relational databases is flexibility, because unlike relational databases that require rigid table schemas and predefined columns, DynamoDB allows flexible data models where attributes can be added or omitted on a per-item basis, making it easier to evolve applications quickly.

Another benefit over relational databases is speed, because it automatically distributes data across partitions and uses optimized key-value access patterns, so reads and writes are extremely fast compared to relational databases that rely on complex joins and indexing.



Table: ContentCatalog - Items returned (6)

Actions

Create item

Scan started on December 11, 2025, 12:00:39

< 1 >

☐	Id (Number)	Authors	ContentType	Difficulty	Price	ProjectCategory
☐	3	[{"S": "Ne...	Project	Easy peasy	0	AI/ML
☐	2	[{"S": "Ne...	Project	Easy peasy	0	Analytics
☐	203		Video		0	
☐	202		Video		0	
☐	201		Video		0	
☐	1	[{"S": "Nat...	Project	Easy peasy	0	Storage



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

